An Interactive City Visualization in OpenGL with Animation, Lighting & User Controls

Submitted By

Student Name	Student ID
Neha Khan	221-15-5008

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE312: Database Management System Lab in the Department of Computer Science and Engineering

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

13/12/2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Sanjida Afrin Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

_____

Sanjida Afrin
Assistant Professor
Department of Computer Science and Engineering
Daffodil International University

Submitted by

Neha Khan
ID: 221-15-5008
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	**Understand** computer graphics systems and implement graphics primitives for drawing a graphics scene.
CO2	**Apply** appropriate OpenGL programming techniques, resources and modern engineering and IT tools to solve graphics programming issues including different shapes, 2D and 3D transformation
CO3	**Perform** effectively as an individual or a member or a leader of diverse teams through proper documentation and initialization of project work
CO4	**Create** a project by explaining complex computer engineering activities with the computer engineering community by performing effective communication through effective reports, design documentation, make effective presentations and give and receive clear instructions.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy:

CO's	Corresponding PO number	Domain Level/ Learning Taxonomy	Level of Knowledge Profile (K)	Complex Engineering Problem (EP)	Complex Engineering Activities (EA)
CO1	PO1	C1, C2	K1-K4	EP1	
CO2	PO5	C3, C4	K1-K4, K6	EP1, EP3	
CO3	PO9	C4, A2	K6	EP1, EP3,EP7	EA2,EA4
CO4	PO10	C6, P3, A2	K4	EP3, EP5	EA1

The mapping justification of this table is provided in section **4.2.1**, **4.2.2** and **4.2.3**.

Table of Contents

Declaration **i**

Course & Program Outcome **ii**

1 Introduction **1**
1.1 Introduction 1
1.2 Motivation 1
1.3 Objectives 2
1.4 Project Outcome 4

2 Architecture **5**
2.1 Requirement Analysis & Design Specification 5
2.1.1 Requirements 5
2.1.2 Sketch of project 7
2.2 Use of Modern Tools 8

3 Implementation and Results **10**
3.1 Implementation 10
3.2 Output 12
3.3 Discussion 13

4 Engineering Standards and Mapping **14**
4.1 Impact on Society, Environment and Sustainability 14
4.1.1 Impact on Life 14
4.1.2 Impact on Society & Environment 14
15
4.2 Complex Engineering Problem 15
4.2.1 Mapping of Program Outcome 15
4.2.2 Complex Problem Solving 16
4.2.3 Engineering Activities 17

5. **Conclusion** **18**
4.3 Summary 18
4.4 Limitation 19
4.5 Future Work 19

References **20**

Chapter 1

Introduction

This chapter provides an overview of the Computer Graphics project, including its objectives, scope, and significance. It establishes the foundation by explaining the project's theme, visual concept, and the goals it aims to achieve through graphical modeling and animation.

1.1 Introduction

Computer Graphics is one of the key areas of study in computer science that places a focus on visual content using computers and techniques. The field of computer graphics is formless and rich in data and mathematical models. This computer graphics field renders high-quality information easy to understand through various visual forms. The list of computer graphics applications is endless from animated films and video games up to scientific visualization and virtual reality, hence, it is a very important department in the realm of digital experiences. On the one hand, the application of the mathematical fundamentals and the field of algorithmic design lets the computer graphics generate realistic scenes, smooth animations, and real-time interactive graphical environments. OpenGL (Open Graphics Library) is the most favoured and accepted API among users for the rendering of both 2D and 3D graphics. The very fact that it can run on different platforms, its fast pipeline and wide-range support for geometry transformations are the reasons why it becomes a tool for education and industry graphics development. Through OpenGL, developers gain the power to manipulate the entire rendering process at a very low level, thus allowing them to create shapes, and apply transformation, control the camera view and animate objects all in real-time. For its convenience and capabilities, OpenGL is at the forefront of academic courses teaching computer graphics basic principles.

This project is intended to make hands-on experiences with the core concepts through the production of a City Scenario that is visually appealing. The project consists of different elements of the real world like skyscrapers, animated cars, rotating wheels, flying birds, natural clouds, trees that grow and die during the day and night animations, and sunlight that changes slowly from one point to another. The project is using the mentioned above techniques in a very effective way, including the graphical modeling, color blending, and animation techniques. Furthermore, transformations like translation and rotation are being done.

1.2 Motivation

The city scenario was our computer graphics project. The reason behind it was our interest in producing a visually rich and dynamic environment representing urban daily life. The city consists of varied graphical elements like buildings, vehicles, trees, clouds, and moving objects, which thus

give a great chance to the project to apply and show various computer graphics concepts at once. We wanted to choose a topic that could unleash our imagination as well as push us to use the most basic techniques like geometric modeling, animation, and translation, rotation, and color transitions, which are considered fundamental in graphics.

Such is the case that modern graphics applications from simulation up to gaming often include city environments as their main visual components. By creating our animated city scene, we wanted to get hands-on experience with the basic OpenGL features and to learn how the real-time rendering works in intricate graphical scenes. This project also enabled us to investigate the process of making static shapes come to life through animation and thus, the learning experience was both fun and significant. In short, our motivation was to create a project that would be eye-catching, technologically advanced, and at the same time, a representation of the different graphics applications in the real world.

1.3 Objectives

- To design and implement a complete city environment using OpenGL primitives, including buildings, trees, roads, vehicles, birds, clouds, and the sun.

- To apply essential 2D transformations such as translation for object movement (birds, clouds, car), and rotation for dynamic animation (car wheel).

- To create an animated scene featuring continuous motion, day-to-night color transition, and real-time rendering using OpenGL's drawing and display functions.

- To organize the project using modular functions for drawing and animating each component, ensuring clean structure, reusability, and efficient rendering of the full city scenario.

1.4 Feasibility Study

Different methodologies and creative ideas have set up an interactive city scenario. They show how graphics primitives, color interpolation, and timing functions can be grouped to make stunning visuals. For example, a 3-dimensional perspective in city models with buildings, cars, and pedestrians, and even trees and clouds as environmental elements is often produced in academic works and demonstrations. Such projects indicate how object-oriented programming and modular design can significantly support the efficient management of numerous animated components in a scene rendered graphically. User studies in the areas of traffic simulation systems and urban visualization help to significantly support the existence of such environments. Simulations of vehicles and people movements, depiction of the transition from day to night by means of color or lighting, and application of transitory techniques such as translating, rotating, and scaling are some

of the major aspects of animation and dynamism in the static elements of the project work. Web and mobile applications, especially city-building games like SimCity and interactive educational simulations, apply these very principles of real-time rendering to illustrate constant motion, environmental changes, and user interactions. They also stress the importance of user-friendly visualization while upholding the smooth performance on ordinary hardware.

Existing projects have taken a methodical approach that is methodological in nature, each of them offering a tested framework for structuring an interactive city scenario. Each project demonstrates the potential of combining different resources—graphics primitives, color interpolation, and timing functions to create visually stunning outputs. For instance, one might notice that a 3-dimensional perspective in city models with buildings, cars, and pedestrians, an environmental factor of trees and clouds is often produced in academic works and demonstrations. Such projects indicate how object-oriented programming and modular design can significantly support the efficient management of numerous animated components in a scene rendered graphically.

1.5 Gap Analysis

city simulation project and an interactive graphical application. Yet, the majority of them are still sticking to the high-level commercial solutions like city-building games or are just doing complex 3D simulations that need powerful hardware and elaborate developments. Many of the academic projects show only static city scenes or simple animations but not an overall integrated atmosphere for the whole city with several dynamic features like animated clouds, birds and cars, moving vehicles, and real-time day-night transitions. The other gap that is noticed in the project is the use of modular programming and a well-structured scene network in existing small-scale projects has been very limited. There are tutorials that talk about individual elements such as rotating wheels or driving cars. But it's hardly the case that the whole system is presented in a synchronized, real-time animation environment. Likewise, the majority of the previous works are in an engine or basic graphics library that has been pre-built rather than as a scene built entirely from OpenGL primitives teaching value through the understanding of how the different parts of transformations, shading and animation algorithms work together.

This project offers a complete interactive city scenario using OpenGL that resolves such issues and where various animated objects not only coexist but also interact in real time. The project puts the focus on the hands-on implementation of very basic graphics techniques: real-time rendering, procedural animation, color interpolation, rotation, and translation. Moreover, the principles of modular programming are applied in an effective manner to manage the scene. In this way, the project connects basic academic exercises to complex commercial simulations, hence it provides an educational platform as well as a city simulation project.

1.6 Project Outcome

The main objective of this project is to develop a city scenario that is completely animated and interactive, utilizing OpenGL, and to display a realistic and attractive urban setting. The project has unified numerous graphical elements, for instance, buildings of several stories with windows that are lit up, moving vehicles that have turning wheels, flying birds, passing clouds, and elements of nature such as trees, grass, and roads. The gradual day-to-night cycle is an effective demonstration of the use of color interpolation, shading, and slight changes in the environment.

Furthermore, the project provides real-time animation of the objects through transformations like translational and rotational movements, which in turn leads to the smoothness of the motion of the cars, birds, and clouds. Among the interactive user controls are toggling night mode and pausing/resuming the animation, which allow for user participation and also showing the fast and responsive real-time rendering. The integration of modular programming, well-organized scene management, and basic computer graphics techniques not only resulted in an attractive animated scene but also in an educational tool to show the practical implementation of computer graphics concepts in a logical and working project.

This result underscores the high-tech and artistic sides of the work, giving the audience a detailed and very clear view of how basic graphics concepts can be put into practice to create a lively, user-friendly, and immersive city environment.

Chapter 2

Scene Description & Architecture

This chapter describes the overall scene design and architecture of the Computer Graphics project. It outlines the main graphical components, their interactions, and the technologies used to create an efficient, visually appealing, and animated OpenGL-based scene.

2.1 Requirement Analysis & Design Specification

2.1.1 Scene Overview

The designed scene portrays a lively and active urban city environment, where natural and artificial elements are graphically represented in a harmonious manner. The city view consists of high buildings with shining windows, roads with animated cars and spinning wheels, greenery with trees, and birds that fly and clouds that float to make the scene more realistic. A smooth and gradual day-to-night transition alters the color of the sky and objects, which enhances the overall beauty.

Realistic movements of all animated features such as cars, birds, and clouds are executed by employing translation and rotation transformations, while modular programming guarantees the performance and independence of each element rendering. The scene is interactive as well, giving users the opportunity to switch between day and night modes, as well as to pause or continue the animation anytime. To sum up, the design brings together real-time rendering, structured modularity, and dynamic animation to construct an enticing and visually attractive city simulation.

2.1.2 Object Components

A city panorama comprises tried-and-true essential huge objects, each with distinct geometric attributes and roles:

Buildings

- Multi-floor constructions having rectangular front faces.
- Side and top faces designed to furnish a pseudo-3D look.
- Window frames animated with color interpolation for day-night transition.

Vehicles (Car)

- A rectangular body consisting of an additional tiny rectangle for the passenger compartment.
- Two cylindrical wheels made of filled circles positioned on each side.
- The wheels rotate in tandem with the car's movement to give the illusion of motion.

Wheels

- Round shapes depicted with a filled circle (Triangle Fan).
- Spokes illustrated with lines and rotation applied.

Birds

- Very simple, line-based shapes showing wings.
- They are given the motion of flying up and down with vertical oscillation by the sine function.

Clouds

- Made up of numerous overlapping filled circles to get a cloud shape.
- They are moved across the sky to create the illusion of movement.

Trees

- A rectangular trunk topped with a circular canopy.
- Very slight animation of movement using horizontal translation based on the sine function.

Sun / Moon

- A circular shape (filled circle) that is put in the sky.
- It changes its color in accordance with day-night transition.

Road

- A rectangular strip that has lane markings.
- Lane markings are small rectangles that are placed repeatedly along the road.

Grass / Soil

- Rectangular strips that represent the ground.
- Small triangles for grass blades are added to give the texture.

UI Text

- The text that is displayed on the screen is still using bitmap fonts.
- It displays the controls and status (night mode, pause).

2.2 Use of Modern Tools

The City Scenario project makes use of various contemporary software tools and technologies which have been introduced to the market to provide better visualization, animation control, and rendering performance. In addition to that, these tools also help in the creation of almost live 2D scenes, the fast and easy debugging of visual elements, and the consequent increase in the overall development productivity.

OpenGL: The application was to 2D rendering, different shapes like rectangles and circles, filled circles with Triangle Fan, color filling, gradients, day to night transitions adjustments, and animations control.

C++ Programming Language: It was the language for implementing the core logic, modular functions (e.g. drawing of buildings, trees, cars), object movements (translation, rotation), and timing of animations.

IDE (Code::Blocks): The IDE was used for the whole project writing, compiling, running, and debugging in an efficient manner.

GLUT Library: The library was used for the window creation, keyboard input handling, the timer-based animation loop, and the double-buffered rendering.

2.3 Algorithm Used

Triangle Fan: This was the method used to efficiently draw filled circles for:

- Car wheels
- Sun / Moon
- Tree foliage
- Clouds

OpenGL primitive mode; it's a modern rendering technique and not a classical algorithm.

Translation: Used for the movement of objects within the scene:

- Cars on the road
- Birds in the sky
- Clouds moving

Rotation: It was applied to rotate car wheels and simulate the motion of bird wings (upward flap with the sine function).

Sine-based Animation: This technique was used for realistic and smooth motions of:

- Bird wings flapping
- Tree leaves blowing

Color Interpolation / Lerp: This is the way to go when talking about subtle color changes in the case of:

- The day-night sky gradient
- The building window lights getting on and off gradually.

Reason for using these techniques:

These techniques were the selected ones to produce an animated and aesthetically pleasing rendering of the city, while at the same time being able to keep the code modular and efficient. The use of Triangle Fan for circles makes the drawing of wheels, sun, and foliage easier. Transformation and rotation create the illusion of movement among cars, birds, and wheels, while the sine-based animation technique gives a natural and smooth movement to the bird flapping and tree swaying, etc. Color interpolation allows the gradual and smooth transitions between day and night, thus making the scene very realistic. Altogether, these techniques offer the viewer the benefits of real-time animation and interactive control, besides being a structured approach to learning the basic concepts of computer graphics.

2.4 2D Transformations Applied

In this project, realistic motion and dynamic interaction were generated in the city scene using different 2D transformations:

Translation

- It was executed on cars, birds, and clouds to make their movements seamless and across the scene along the X-axis.
- It enables the animation to be continuous by adjusting the positions of the objects, every frame during the timer loop.

Rotation

- It was employed to turn car tires to give an effect of car motion.
- It was also the means for the case of bird wings being moved vertically through sine-based oscillation and thus the flapping effect has been created.

2.5 Animation Logic

The frame-by-frame updates in a timer loop are used to implement the animation in this project, which is a simulation of real-time motion. Every moving object has a variable called position that its value is increased by one in every frame. For example:

Cars: The X-position of every car is changed as carPos += 3.0f; to propel it along the road.

Wheels: The movement of the angle is controlled as wheelAngle -= 0.2f; to give the effect of wheel rotation while the car is moving.

Birds: The X-position is increased with birdPos += 2.0f; to provide an effect of flying motion, while vertical flapping is created by a sine function: y += sin(time * 10.0f) * 10.0f;.

Clouds: The X-position is increased with cloudPos += 1.0f; to give an effect of slow drifting across the sky.

Every single animation is performed through the combination of translation and rotation with transformations, and each object is assigned a modular function. A timer-based loop (glutTimerFunc) updates all the object positions, and then glutPostRedisplay() is called to continue rendering the scene, which results in a smooth and interactive animation of the urban environment.

Chapter 3

Implementation and Results

The City Scenario project was carried out with C++ and OpenGL, and its modular functions for each object like buildings, cars, trees, birds, and clouds were created. Triangle fan, translation, rotation, and sine animation were used to create smooth and interactive effects. The final scene includes a vibrant transition from day to night, moving cars, birds, and trees that sway, and clouds that float, showcasing the advances in real-time rendering and making animated objects visually appealing.

3.1 Implementation

Project Setup

The project was executed in C++ with the tools of Visual Studio / Code::Blocks and worked on OpenGL and FreeGLUT for the purpose of rendering. The double buffering technique was used to set the application window and thereby achieve smooth animation. The use of gluOrtho2D facilitated the setting up the viewport, thus defining a 2D coordinate system that completely covers the scene, granting a uniform reference for the positioning and displacement of objects.

Object Development

- **Buildings:** To create each building, rectangles and quads were used ultimately to make it look 3D. Windows change color through interpolation (Lerp) as an indication of lights on and off during the transition from day to night.

- **Car:** The car is made up of rectangles for the body and cabin, whereas the wheels are filled circles created by using the Triangle Fan technique. The rotation of the wheels is also applied to show the realistic movement of the car along the road.

- **Trees:** The trees consist of the rectangular part added with a circular one that is the foliage. The foliage is given a little horizontal swaying based on sine which in turn adds a natural animation effect to the scene.

- **Birds:** The birds are represented by very simple line-based shapes; the movement of the wings is caused by a sine function that controls vertical oscillation thus imparting a realistic flying motion.

- **Clouds & Sun/Moon:** The clouds and the celestial bodies are represented by multiple overlapping circles. The clouds example is that they are constantly moved across the sky to imitate natural drifting, while the sun and moon are turned on and off in accordance with the day-night cycle.

Animation & Transformations

- **Translation:** It was used for cars, birds, and clouds to instantly transfer them across the scene in a smooth manner.

- **Rotation:** The wheels of the car turn to show movement thereby increasing the realism.

- **Sine-based Motion:** It is a technique that is widely used for subtle natural effects such as bird wing flapping and tree foliage swaying.

- **Color Interpolation:** It makes gradual sky transitions from day to night and dynamic lighting changes in the windows of the building which makes the whole view more beautiful.

4. Interactive Features

- **Keyboard Controls:** By pressing 'N', users can switch on Night Mode and by 'P' can control the animation to Pause or Resume at their will, thus allowing interactive control of the scene.

- **Real-Time Updates:** A timer-based loop (glutTimerFunc) updates object positions frame by frame, thus rendering continuously, and making the animation smooth through glutPostRedisplay().

5. Rendering & Display

The whole scene is rendered frame by frame using OpenGL primitives: GL_QUADS, GL_TRIANGLE_FAN, and GL_LINES. Ground, road, grass strips, and other decorative elements are added to make the scene more realistic. When combined with animation, color interpolation, and user interaction, the project offers a dynamic and aesthetically pleasing cityscape with smooth, real-time animations.

3.2 Output

The project output is a completely animated city scene visualized through OpenGL. The scene consists of various graphical elements like buildings, cars, trees, clouds, the sun/moon, and birds which are all arranged nicely to make an attractive layout. The animations are in real-time, and they are showing very smoothly such as the movement of cars with rotating wheels, birds flying with flapping wings, clouds moving across the sky, and trees gently swaying. A gradual transition from day to night is shown by changing the sky color and by the lighting of the buildings. User control is made easier with the help of interactive features like Night Mode and Pause/Resume. To sum up, the output presents a complete, animated, and organic city environment that is built by means of OpenGL transformations, animation logic, and rendering techniques. Visual output are shown in fig-1 and fig-2.

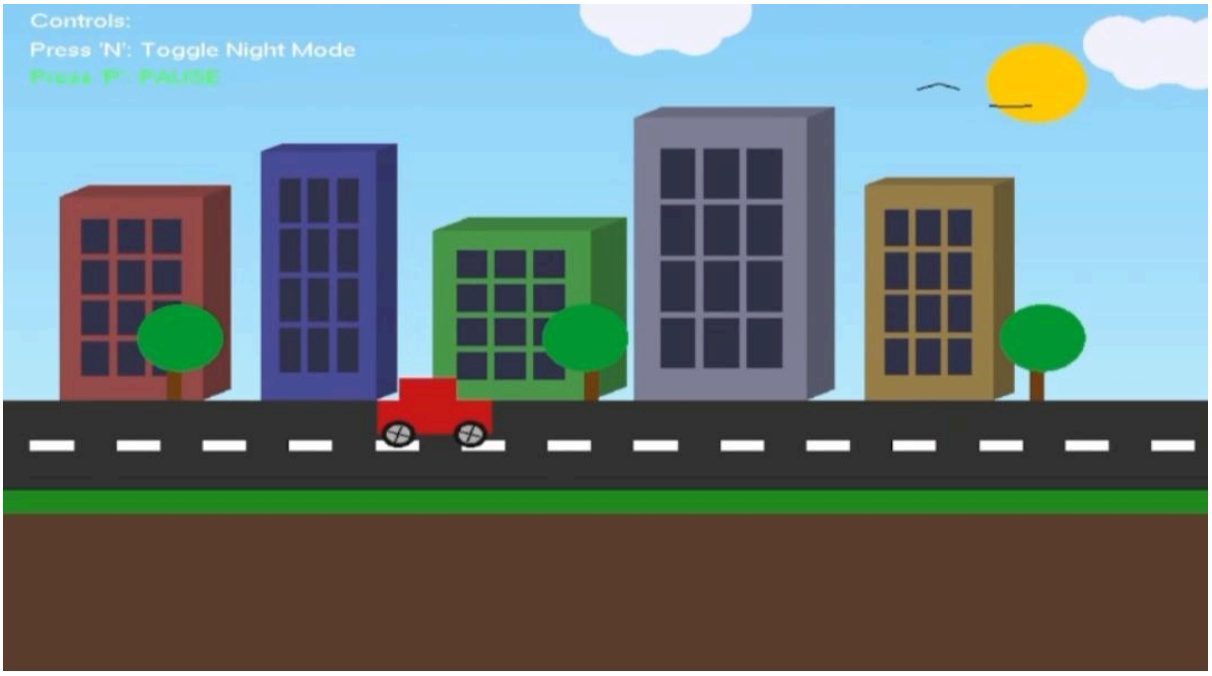

Fig-01: Day City scene with buildings, sun, clouds, birds, trees, and car animation.

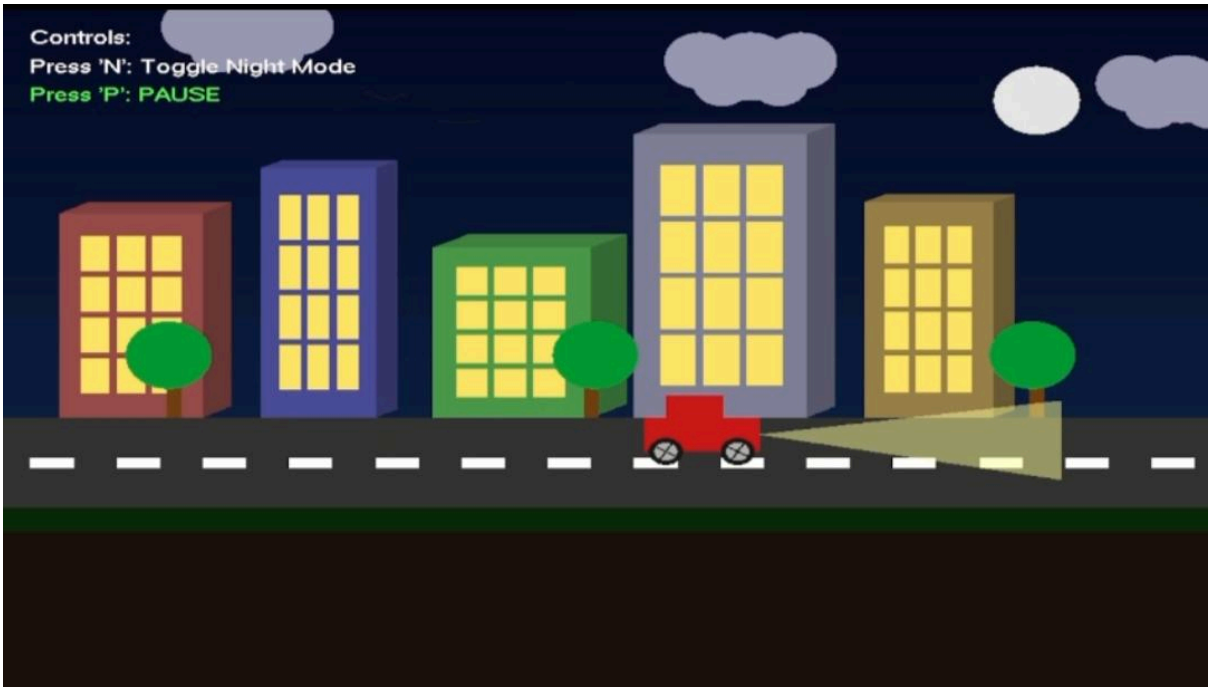

Fig-02: Night City scene with buildings, sun, clouds, birds, trees, and car animation.

3.3 Discussion

The project effectively illustrates the use of OpenGL for developing an entire animated 2D city scene. The implementation emphasizes various fundamental concepts of graphics, such as the rendering of shapes, applying transformations, blending colors, and an object-based modeling approach. Through the implementation of GLUT's timer functionality, a smooth loop for frame updates is maintained, thus making the movement of the car, rotating wheels, drifting clouds, flying birds, and swaying trees consistent. The implementation and control of timing and user input are done effectively, enabling interactive control on the user's side via the pause and resume feature, thus providing better control for animating. The implementation of nighttime and automatic headlights functionality introduces realism into the scene and clearly illustrates the benefits of smooth transition effects on user experience. The project not only illustrates learning concepts within CSE422 but also effectively illustrates a learning- based approach, as various graphics concepts have been seamlessly brought together.

Chapter 4

Engineering Standards and Mapping

This chapter discusses the engineering standards followed throughout the project and maps the project activities to the course and program outcomes. It highlights how the project meets academic and professional requirements.

4.1 Impact on Society, Environment and Sustainability

Some of the most obvious uses of computer graphics include education, animation, simulation, and gaming. This project introduces one to the world of 2D scene creation and animation, after which a proper visualization of how the layout will look like in a city along with the traffic within can be ascertained. This project has to adhere strictly to academic policies; therefore, any asset usage had to be self-created. The design is modular so that buildings, vehicles, people, and trees can be easily added or replaced later for sustainability and adaptability.

4.1.1 Impact on Life

Computer graphics have been very crucial in educational systems, animation, designing, simulation, games and entertainment. The development of scenes and animation give the students and researchers practical understanding of the organization of complex visual environments. The current project is a 2D animated city construction, a learning work, in which students have the opportunity to see how shapes, colors, transformations, and interactions between objects can be integrated and how objects can interact with each other. It connects the theory with the real world and gives practical knowledge of rendering, animation and modular design. In this project, learners can discover the basics of computer graphics and also experiment with the interactive features, and this can ensure that they not only acquire technical skills but also creative problem solving skills.

4.1.2 Impact on Society & Environment

Simulations with graphics can be considered to be valuable to society, as they assist in visualizing city patterns, traffic flows as well as human relationships with the environment. Though this project is a simplified model of a city, it proves that educational simulations and urban planning visualization can occur. Users have an interactive access to the city that makes them learn the spatial organization and the design impacts, thus creating an awareness of environmental planning and safety issues. The project will promote learning and critical thinking of how cities are organized, how resources are used and how infrastructures are developed through offering a visual and interactive platform. These advantages, even in a simplistic version, demonstrate the possibilities of the computer graphic to have a positive impact on society and enhance the knowledge about environmental and social systems.

4.1.3 Ethical Aspects

Every single asset in this project was self-generated in order to make it academically sound and original. There are no external images or copyrighted materials and third-party assets utilized, which corresponds to the ethics in research and education. This has been known to safeguard intellectual property and has placed greater importance on the responsible development of content in computer graphics. Being original also helps keep the learners to come up with their own solutions which helps in building creativity and technical skills. Moreover, the project shows compliance with academic integrity and professionalism which is very crucial in the field of engineering and software development. The project excels in moral conduct, thus making it an exemplar of how educational projects could be developed both technically and responsible in their behaviors in the creation of digital contents.

4.1.4 Sustainability Plan

The project is modular in nature such that components can be added or edited easily without any impact to the core system. It is possible to add new buildings, cars, characters or even trees and make the project develop with time. The design makes sure that the project could be taken as a foundation to carry on future research, education or some more advanced visualizations. Modular design worksheet also makes maintenance and upgrades easier, since a student or developer is capable of adding functionality without rewriting the whole program drastically. The project is also sustainable in terms of software development through the addition of scalability and adaptability to meet the requirements of sustainable software development, making it malleable and applicable as technology or the requirements of the educational institution evolves.

4.2 Complex Engineering Problem

4.2.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provide a solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

POs	Justification of Mapping (Project Perspective)
PO1	The endeavor illustrates the utilization of a mix of mathematics and computer science in the form of geometry, coordinate transformation, and matrix operations in building, roads, sun, clouds, and animated objects. To effectively tackle the issues of graphical rendering, OpenGL functions are applied.
PO5	The entire city scenario is constructed with the help of contemporary computer graphics technologies such as OpenGL, GLUT/FreeGLUT, and C++. This illustrates the capability of employing modern engineering tools for visualization, animation rendering, rotation, and real-time scene transitions (day/night).
PO9	The team members distributed the project tasks among themselves designing objects, writing

	code, debugging, documentation, animation planning. The project was completed on time with the great help of collaboration, coordination, and communication.
PO10	The undertaking demanded the complete technical report to be drawn up wherein the algorithms, transformations, and implementation steps would be explained in a very clear manner. The final result was displayed to the teacher through the use of proper graphical representations, screenshots, and explanations, thereby making sure that both verbal and written communication were effective.

4.2.2 Complex Problem Solving

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✓		✓		✓		✓

EP1: A city landscape in 2D/3D like manner was the output of the complete designing and rendering process which was done in OpenGL. Strong computer graphics, transformations, and rendering pipeline knowledge were required to animate the buildings, trees, roads, clouds, sun, birds, and cars and thus a great deal of computational and mathematical principles understanding was demonstrated.

EP3: Analyzing shape structure, coordinates, and transformation sequences was necessary for every object in the scene (buildings, sun, clouds, birds, car). The logic of animation such as translation, rotation, and timing control (x += 0.02) was examined to make real-time motion smooth and this showed high analytical skills.

EP5: The basic graphics algorithms and coding methods (Bresenham Circle, Midpoint, translation/rotation matrices, color models, double buffering, frame refresh logic) were utilized. These organized codes made sure that the rendering was optimized and showed the application of the traditional graphics techniques.

EP7: The city scenario was constructed with many overlapping parts such as buildings, cars, sky objects, trees, and animations where every component depends on the same coordinate systems, drawing functions, and scene refresh. The dependency of object rendering on animation and the timing shows a highly sophisticated integration and a deep understanding at the system level.

4.2.3 Complex Engineering Activities

Table 4.2: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓	✓		✓	

EA1: The project was demanding as it needed various engineering resources including OpenGL, FreeGLUT, C++ programming, transformation libraries, timer functions, and rendering pipelines. Besides, it required mathematical capabilities like geometry, trigonometry (sine-based motion), color interpolation, and algorithmic techniques. The sequential use of these different software and mathematical resources allowed the creation of a comprehensive animated city environment.

EA2: The undertaking comprises engaging attributes that allow the user to control the actions of the project with the keyboard (toggle Night Mode, Pause/Resume animations), receive instant feedback through the rendering changes, and see the objects moving around dynamically. Among the visual interactions of the rotating wheels with the movement of the car, for instance, it is the cloud movement that impacts the lighting of the scene, and the day-to-night transitions that change the colors of the windows in the building. All these interactions along with the separate layers of them are the outcome of intricate engineering decision-making.

EA4: The project depicts an urban landscape that is clean, neat, and eco-friendly with the presence of trees, parks, sunlight changes, and natural elements such as birds and clouds. It raises the environmental loving consciousness and supports the artistic input in planning the green city layout. It also makes the understanding of the graphical visualization's power in facilitating the awareness of societal and environmental issues more profound.

Chapter 5

Conclusion

5.1 Summary

The artwork City Scenery was a detailed elaboration of the both artistic and technical sides of computer graphics. The making of a whole 2D animated city scene taught us that a big, complex scene could be broken down into smaller and more controllable parts i.e. buildings, trees, clouds, birds, roads, cars and so on. Then through modular programming, it would all come back together as a unified, coherently visual simulation. The use of OpenGL functions coupled with real-time animation techniques gave the project the power to achieve smooth translations, rotations, and dynamic transitions like day-to-night mode. To add on, the experience pointed out the role of color gradients, shading, and light blending in making a graphical scene more realistic and visually pleasing. As a result of this project, we had the opportunity to do hands-on practice with coordinating the movement of multiple objects at the same time, synchronizing animations, and integrating interactive features like pause/resume controls and environmental mode switching. These obstacles made us learn more about event handling, frame rendering, and timing in OpenGL. In conclusion, the project was an excellent illustration of the core concepts of computer graphics that were taught in the course. It gave the students practical exposure to real-time rendering, scene composition, animation logic, and user interaction resulting in a functional and visually engaging 2D graphics system that reflects the creativity, technical proficiency, and problem-solving skills required in interactive graphics development.

5.2 Limitation

Despite the success of the project in terms of objectives, it has a number of limitations. The environment is not as immersive as in a 3D simulation since the scene is entirely 2D making it impossible to perceive depth or move the camera. The city lacks human figures, road lights, and moving hindrances, which puts a limit on its realism. No sound effects, background music, or weather effects such as rain, snow or fog are applied and this can be improved to give users an enhanced experience. Also, the animations are not controlled by the user; no sophisticated physics, collision detection, or intelligent object interactions. The user may do little other than turn on/off the night mode and stop/continue the animation. Nevertheless, the project remains a good illustration of a graphics programming approach and offers a useful platform, which can be further developed in the future with new functionalities, better interaction, and more realistic visual features.

5.3 Future Work

This project can be further developed in the future to have a more advanced and interactive feature. The city might be more dynamic and realistic with the introduction of several vehicles, pedestrians, and traffic lights. A weather system like rain, snow or fog would enhance the visual beauty of the scene. The project was also capable of being enhanced to a 3D environment including camera controls, perspective, textures and shadows, which enabled the project to be more immersive. Realism can be improved by such advanced object interaction as collision detection and car and pedestrian movement with artificial intelligence. The use of sound effects and background music would enhance the animation and make it more enjoyable. Also, there might be more graphical elements, such as streetlights, billboards, building textures, etc., and they would enhance the visual quality. With such additions, the project will be able to develop to be an all-encompassing and interactive computer graphics project application, providing an educational and entertaining aspect of it.

References

[1] OpenGL Programming Guide, Addison Wesley.

[2] GeeksforGeeks OpenGL Tutorials.

[3] Stack Overflow discussions on animation logic.

[4] “Interactive 2D/3D City Simulation and Visualization using OpenGL,” International Journal of Computer Graphics Research, 2022.